

第 8 章

IO 库

导读

本章介绍了处理面向流的输入输出的 C++ 标准库类：

- `iostream` 处理控制台 IO
- `fstream` 处理命名文件 IO
- `stringstream` 完成内存 `string` 的 IO

本章的练习主要帮助读者熟悉基本的控制台输入输出、文件输入输出和字符串输入输出，以及条件状态检测和设置等。

练习 8.1：编写函数，接受一个 `istream&` 参数，返回值类型也是 `istream&`。此函数须从给定流中读取数据，直至遇到文件结束标识时停止。它将读取的数据打印在标准输出上。完成这些操作后，在返回流之前，对流进行复位，使其处于有效状态。

【出题思路】

本题是流的输入输出的基本练习。此外，本节介绍了流的条件状态，本题还对流的结束状态（本题是遇到文件结束符）、错误状态和数据错误状态（例如要求输入整数时输入了字符）的检测和处理进行了练习。

【解答】

```
#include <iostream>
#include <stdexcept>
using namespace std;

istream & f(istream & in)
{
    int v;
```

```

while (in >> v, !in.eof()) { // 直到遇到文件结束符才停止读取
    if (in.bad())
        throw runtime_error("IO 流错误");
    if (in.fail()) {
        cerr << "数据错误, 请重试: " << endl;
        in.clear();
        in.ignore(100, '\n');
        continue;
    }
    cout << v << endl;
}
in.clear();
return in;
}

int main()
{
    cout << "请输入一些整数, 按 Ctrl+Z 结束" << endl;
    f(cin);
    return 0;
}

```

练习 8.2: 测试函数, 调用参数为 `cin`。

【出题思路】

通过运行程序来观察流的状态如何产生变化。

【解答】

运行过程如下所示:

请输入一些整数, 按 Ctrl+Z 结束

1 2 3

1

2

3

7.6

7

数据错误, 请重试:

a

数据错误, 请重试:

4

4

^Z

Process returned 0 (0x0) execution time : 20.635 s
Press any key to continue.

练习 8.3: 什么情况下, 下面的 `while` 循环会终止?

```
while (cin >> i) /* ... */
```

【出题思路】

进一步理解流的状态的检测方式。

【解答】

遇到了文件结束符，或者遇到了 IO 流错误，或者读入了无效数据。

练习 8.4: 编写函数，以读模式打开一个文件，将其内容读入到一个 string 的 vector 中，将每一行作为一个独立的元素存于 vector 中。

【出题思路】

本题练习文件输入和流的逐行输入，还练习了使用迭代器遍历容器中的元素。

【解答】

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>

using namespace std;

int main()
{
    ifstream in("data");           // 打开文件
    if (!in) {
        cerr << "无法打开输入文件" << endl;
        return -1;
    }

    string line;
    vector<string> words;
    while (getline(in, line)) {     // 从文件中读取一行
        words.push_back(line);     // 添加到 vector 中
    }

    in.close();                    // 输入完毕，关闭文件

    vector<string>::const_iterator it = words.begin(); // 迭代器
    while (it != words.end()) {    // 遍历 vector
        cout << *it << endl;      // 输出 vector 中的元素
        ++it;
    }

    return 0;
}
```

【其他解题思路】

vector 的遍历还可使用范围 for 循环来实现。

练习 8.5: 重写上面的程序，将每个单词作为一个独立的元素进行存储。

【出题思路】

本题练习逐个数据的输入方式。

【解答】

将第 18 行的 `while (getline(in, line))` {改为 `while (in >> line)` {即可。

练习 8.6: 重写 7.1.1 节的书店程序 (第 229 页), 从一个文件中读取交易记录。将文件名作为一个参数传递给 `main` (参见 6.2.5 节, 第 196 页)。

【出题思路】

通过一个较大的例子继续练习文件输入, 并练习从命令行获取参数及参数合法性的检测。

【解答】

```
#include <iostream>
#include <fstream>
#include "Sales_data.h"

using namespace std;

int main(int argc, char *argv[])
{
    if (argc != 2) {
        cerr << "请给出文件名" << endl;
        return -1;
    }
    ifstream in(argv[1]);
    if (!in) {
        cerr << "无法打开输入文件" << endl;
        return -1;
    }

    Sales_data total;                                // 保存当前求和结果的变量
    if (read(in, total)) {                            // 读入第一笔交易
        Sales_data trans;                            // 保存下一条交易数据的变量
        while(read(in, trans)) {                    // 读入剩余的交易
            if (total.isbn() == trans.isbn()) // 检查 isbn
                total.combine(trans);              // 更新变量 total 当前的值
            else {
                print(cout, total) << endl;         // 输出结果
                total = trans;                       // 处理下一本书
            }
        }
        print(cout, total) << endl;                 // 输出最后一条交易
    } else {                                          // 没有输入任何信息
        cerr << "没有数据" << endl;                // 通知用户
    }

    return 0;
}
```

练习 8.7: 修改上一节的书店程序，将结果保存到一个文件中。将输出文件名作为第二个参数传递给 main 函数。

【出题思路】

本题练习文件输出。

【解答】

```
#include <iostream>
#include <fstream>
#include "Sales_data.h"

using namespace std;

int main(int argc, char *argv[])
{
    if (argc != 3) {
        cerr << "请给出输入、输出文件名" << endl;
        return -1;
    }
    ifstream in(argv[1]);
    if (!in) {
        cerr << "无法打开输入文件" << endl;
        return -1;
    }

    ofstream out(argv[2]);
    if (!out) {
        cerr << "无法打开输出文件" << endl;
        return -1;
    }

    Sales_data total;
    if (read(in, total)) {
        Sales_data trans;
        while(read(in, trans)) {
            if (total.isbn() == trans.isbn())
                total.combine(trans);
            else {
                print(out, total) << endl;
                total = trans;
            }
        }
        print(out, total) << endl;
    } else {
        cerr << "没有数据" << endl;
    }

    return 0;
}
```

// 保存当前求和结果的变量
// 读入第一笔交易
// 保存下一条交易数据的变量
// 读入剩余的交易
// 检查 isbn
// 更新变量 total 当前的值
// 输出结果
// 处理下一本书
// 输出最后一条交易
// 没有输入任何信息
// 通知用户

练习 8.8: 修改上一题的程序, 将结果追加到给定的文件末尾。对同一个输出文件, 运行程序至少两次, 检验数据是否得以保留。

【出题思路】

本题练习文件的追加模式。

【解答】

将上一题程序的第 19 行 `ofstream out 1(argv[2]);` 改为 `ofstream out(argv[2], ofstream::app);`。

练习 8.9: 使用你为 8.1.2 节 (第 281 页) 第一个练习所编写的函数打印一个 `istringstream` 对象的内容。

【出题思路】

本题练习字符串流的输入。

【解答】

```
#include <iostream>
#include <sstream>
#include <string>
#include <stdexcept>

using namespace std;

istream & f(istream & in)
{
    string v;
    while (in >> v, !in.eof()) {           // 直到遇到文件结束符才停止读取
        if (in.bad())
            throw runtime_error("IO 流错误");
        if (in.fail()) {
            cerr << "数据错误, 请重试: " << endl;
            in.clear();
            in.ignore(100, '\n');
            continue;
        }
        cout << v << endl;
    }
    in.clear();
    return in;
}

int main()
{
    ostringstream msg;
    msg << "C++ Primer 第五版" << endl;
    istringstream in(msg.str());
    f(in);
    return 0;
}
```

练习8.10:编写程序,将来自一个文件中的行保存在一个 `vector<string>` 中。然后使用一个 `istream` 从 `vector` 读取数据元素,每次读取一个单词。

【出题思路】

本题继续练习字符串流的输入。

【解答】

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>

using namespace std;

int main()
{
    ifstream in("data");                // 打开文件
    if (!in) {
        cerr << "无法打开输入文件" << endl;
        return -1;
    }

    string line;
    vector<string> words;
    while (getline(in, line)) {          // 从文件中读取一行
        words.push_back(line);          // 添加到 vector 中
    }

    in.close();                          // 输入完毕, 关闭文件

    vector<string>::const_iterator it = words.begin(); // 迭代器
    while (it != words.end()) {          // 遍历 vector
        istringstream line_str(*it);
        string word;
        while (line_str >> word)         // 通过 istringstream 从 vector 中读取数据
            cout << word << " ";
        cout << endl;
        ++it;
    }

    return 0;
}
```

练习8.11: 本节的程序在外层 `while` 循环中定义了 `istringstream` 对象。如果 `record` 对象定义在循环之外, 你需要对程序进行怎样的修改? 重写程序, 将 `record` 的定义移到 `while` 循环之外, 验证你设想的修改方法是否正确。

【出题思路】

本题练习字符串流的重复使用, 每次通过 `str` 成员将流绑定到不同的字符串,

同时还要调用 `clear` 来重置流的状态。

【解答】

```
#include <iostream>
#include <sstream>
#include <string>
#include <vector>

using namespace std;

struct PersonInfo {
    string name;
    vector<string> phones;
};

int main()
{
    string line, word;           // 分别保存来自输入的一行和单词
    vector<PersonInfo> people;    // 保存来自输入的所有记录
    istringstream record;

    while (getline(cin, line)) {
        PersonInfo info;         // 创建一个保存此记录数据的对象
        record.clear();           // 重复使用字符串流时，每次都要调用 clear
        record.str(line);         // 将记录绑定到刚读入的行
        record >> info.name;      // 读取名字
        while (record >> word)    // 读取电话号码
            info.phones.push_back(word); // 保持它们

        people.push_back(info);   // 将此记录追加到 people 末尾
    }

    return 0;
}
```

【其他解题思路】

还可通过 `rddbuf` 成员函数来获得 `record` 流的缓冲区（一个 `stringbuf` 对象的指针），然后利用 `stringbuf` 的 `str` 成员函数来直接设置缓冲区（字符串）内容。注意，此方法仍然需要调用 `clear` 来将流复位。

练习 8.12：我们为什么没有人在 `PersonInfo` 中使用类内初始化？

【出题思路】

体会根据应用特点决定程序设计策略。

【解答】

由于每个人的电话号码数量不固定，因此更好的方式不是通过类内初始化指定人名和所有电话号码，而是在缺省初始化之后，在程序中设置人名并逐个添加电话号码。

练习 8.13: 重写本节的电话号码程序，从一个命名文件而非 cin 读取数据。

【出题思路】

本题练习文件流和字符串流输入输出的综合应用。

【解答】

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>

using namespace std;

struct PersonInfo {
    string name;
    vector<string> phones;
};

string format(const string &s) { return s; }

bool valid(const string &s)
{
    // 如何验证电话号码将在第 17 章介绍
    // 现在简单返回 true
    return true;
}

int main(int argc, char *argv[])
{
    string line, word;                // 分别保存来自输入的一行和单词
    vector<PersonInfo> people;        // 保存来自输入的所有记录
    istringstream record;

    if (argc != 2) {
        cerr << "请给出文件名" << endl;
        return -1;
    }
    ifstream in(argv[1]);
    if (!in) {
        cerr << "无法打开输入文件" << endl;
        return -1;
    }

    while (getline(in, line)) {
        PersonInfo info;              // 创建一个保存此记录数据的对象
        record.clear();               // 重复使用字符串流时，每次都要调用 clear
        record.str(line);             // 将记录绑定到刚读入的行
        record >> info.name;          // 读取名字
        while (record >> word)        // 读取电话号码
            info.phones.push_back(word); // 保持它们

        people.push_back(info);       // 将此记录追加到 people 末尾
    }
}
```

```

    }

    ostream os;
    for (const auto &entry : people) {           // 对 people 中每一项
        ostream formatted, badNums;             // 每个循环步创建的对象
        for (const auto &nums : entry.phones) {   // 对每个数
            if (!valid(nums)) {
                badNums << " " << nums; // 将数的字符串形式存入 badNums
            } else
                // 将格式化的字符串“写入” formatted
                formatted << " " << format(nums);
        }
        if (badNums.str().empty())                // 没有错误的数
            os << entry.name << " "              // 打印名字
                << formatted.str() << endl;       // 和格式化的数
        else                                       // 否则，打印名字和错误的数
            cerr << "input error: " << entry.name
                << " invalid number(s) " << badNums.str() << endl;
    }
    cout << os.str() << endl;

    return 0;
}

```

练习 8.14：我们为什么将 `entry` 和 `nums` 定义为 `const auto&`?

【出题思路】

回顾范围 `for` 语句的相关内容。

【解答】

这两条语句分别使用范围 `for` 语句枚举 `people` 中所有项（人）和每项的 `phones` 中的所有项（电话号码）。使用 `const` 表明在循环中不会改变这些项的值；`auto` 是请求编译器依据 `vector` 元素类型来推断出 `entry` 和 `nums` 的类型，既简化代码又避免出错；使用引用的原因是，`people` 和 `phones` 的元素分别是结构对象和字符串对象，使用引用即可避免对象拷贝。